

PERF-002 — cross-encoder score-preserving latency audit

Read-only · no patch applied · E-L10 not validated, not frozen · holdout not opened

Source audited: `grok/ce-latency-handoff` @ **7661ac5**, and re-verified byte-identical against the full tree `grok/v2-dev` @ **b044b41** — so nothing here rests on the handoff pack being a faithful copy. CE artifact sha256 re-verified against the preregistered value. Model confirmed `BertForSequenceClassification`, `logits [batch,1]`, 6 layers / hidden 384 — MiniLM-L6.

TWO RESULTS, POINTING OPPOSITE WAYS

- 1. **Grok's bitwise-identity claim is correct. I doubted it and I was wrong.** All 9 levers tested return **bit-identical logits** — including width 24 versus width 512. So the equivalence criterion here is **bitwise**, not a tolerance, and ranks are identical by construction.
- 2. **fast=True is the wrong bundle.** It couples a machine-independent **2.07×** win with a machine-dependent `threads=8` that is a **0.68×** regression on this 4-CPU host. Bundled it delivers only 1.22×. **The two levers must be separated.**

98.9%
of CE time is ONNX inference

53.6%
of tokens fed to it are `[PAD]`

1.12%
is everything else combined

2.07×
bitwise, machine-independent

B — Measured latency decomposition

104 pairs (E-L10 mean union 104.1), one query, warmed, this container:

Component	baseline pad=512	share	bucketed pad=batch	share
pre-pass tokenize (bucketing only)	0.0	0.00%	49.8	0.89%
tokenize	94.3	1.03%	104.1	1.87%
numpy pack	8.6	0.09%	5.9	0.11%
ONNX inference	9,087.0	98.87%	5,408.3	97.12%
postprocess / unpermute	0.4	0.00%	0.4	0.01%
batching / loop overhead	0.2	0.00%	0.2	0.00%
total	9,190		5,569	

Batch widths — baseline `[512 ×7]`; bucketed `[34, 86, 168, 310, 512, 512, 512]`. Session and tokenizer are built **once per process** (859 ms), correctly excluded from per-query cost.

ABSOLUTE MILLISECONDS HERE ARE NOT EXP-018B'S

This box is **~1.55×** slower than the EXP-018B host (4 CPUs, Intel Xeon @2.80GHz, AVX-512 + VNNI). **Ratios transfer; milliseconds do not.** Every projection below is built from ratios only.

C — Dominant bottleneck

ONNX inference on padding. 98.87% of CE time is inside `session.run`, and **53.6% of the tokens fed to it are `[PAD]`**. Attention is $O(n^2)$ in width, so a 172-token pair padded to 512 costs roughly 3× the FFN work and ~9× the attention work of the

same pair at its true width.

Measured over 400 cs_v1_control passages, synthetic 19-token query: mean **237.8**, p50 172, p90 512, **26.2% already at 512** — independently reproducing Grok's 237 / 26.3% on different hardware.

A FINDING, NOT AN OMISSION

Brief items 3, 4, 7 and 8 — tokenization, tensor construction, query-side caching, candidate-side caching — sum to **1.12% of CE time**. Perfect elimination of all of it saves about 100 ms of 5904. **These are not levers.**

D — Score-preserving optimizations, ranked

Change	ms	speedup	bitwise	machine-dependent?
baseline <code>CrossEncoderReranker()</code>	8,638	1.00×	—	—
D1 — bucketed <code>pad="batch"</code> , threads unchanged	4,168	2.07×	✓	no
D1b — same, <code>batch_size=1</code>	3,681	2.35×	✓	probably
D2 — <code>threads=8</code> alone	12,686	0.68×	✓	YES
<code>fast=True</code> (bundles D1 + <code>threads=8</code>)	7,096	1.22×	✓	YES
<code>pad="batch"</code> unsorted	8,161	1.06×	✓	no
<code>batch_size=1</code> at pad 512	7,942	1.09×	✓	probably

D1 — ship this one. 2.07× here, and ~2.16× derived from Grok's own 5571 / 3728 / 1729 triple. Agreement across two different CPUs is what makes it safe to project.

D3 — merge the two CE calls. 94 and 10 are bucketed separately; merging raises the bucketing ceiling (`run_exp018b.py:391` and `:409`). Bitwise-safe, because batch composition provably does not affect logits.

D2 — do not ship in a bundle. `batch_size=1` with bucketing was a further 14% here was a further finding Grok did not test (2.35×), worth one measurement on the target box before adopting.

E — Equivalence: bitwise is achievable

Compared on exact bit patterns (`struct.pack("<d", x)` , so `-0.0` would show as a difference and NaN would be visible), not a tolerance:

Configuration vs <code>CrossEncoderReranker()</code>	identity	max Δ	ranks
batch 1 (pad=512, t4)	✓ bitwise	0.000e+00	same
batch 32 (pad=512, t4)	✓ bitwise	0.000e+00	same
threads=8 (pad=512, b16)	✓ bitwise	0.000e+00	same
threads=1 (pad=512, b16)	✓ bitwise	0.000e+00	same
pad=batch unsorted (t4, b16)	✓ bitwise	0.000e+00	same
pad=batch bucketed (t4, b16)	✓ bitwise	0.000e+00	same
pad=batch bucketed b1 (t4)	✓ bitwise	0.000e+00	same
fast=True (t8, pad=batch, bucketed)	✓ bitwise	0.000e+00	same
pad width 24 vs 512 (8 shortest pairs)	✓ bitwise	0.000e+00	same

WHY IT HOLDS — NOT LUCK

`attention_mask` sets padded positions to exactly `0.0` after softmax, and adding exact zeros to a float accumulation in sequence order changes nothing. MLAS parallelizes GEMM over M/N tiles rather than splitting the K reduction, so each output element's summation order is fixed regardless of thread count or batch shape. **Neither padding nor threading perturbs a single bit** — so ranks are identical by construction and the `(-score, a_rank, chunk_id)` tie-break is never exercised differently.

The gate to run before flipping anything

1. bitwise logit identity on ≥ 104 real E-L10 pairs (bit patterns, not isclose)
2. ordered candidate identity after un-permutation
3. final top-10 rank identity
4. exact metric identity 44/50, 40/50, .8000, .5956, .9200, union 104.1
5. determinism in-process and cross-process
6. re-run the gate on the deployment host

Honest limitation: EXP-018B stores no CE logits, so there is no stored-logit replay gate. Gate 1 must run as a paired A/B in one process.

G — Risks

G3B — SEVEN CALLERS SHARE THE DEFAULTS, TWO ALREADY FROZEN

```
run_exp015_development.py:355 , run_exp016_development.py:420 , run_exp018_development.py:231 ,  
run_exp018_v2devset001.py:164 , run_exp018b.py:341 , run_eval_val_002.py:526 ,  
run_eval_holdout_001.py:582 .
```

changing the class default would silently re-path EVAL-VAL-002 and EVAL-HOLDOUT-001; bitwise identity keeps the numbers but breaks provenance. Enable per call site; do not change the class defaults.

CORRECTION — G2, AFTER THE FULL TREE LANDED

my first pass said no artifact substantiated Grok's 8 cores; ce-latency-microbench.json records `cpu_count=8`. I looked in EXP-018B-results.json's environment block, which does not record it. The finding survives: `threads=8` is validated for that host and regresses 32% on this 4-CPU one, so it stays a per-host setting rather than a portable default.

G1 — bitwise identity is a property of this ORT build on this CPU class, not a theorem. It held on two independent machines at ORT 1.29.0. A different ORT version or a CPU without AVX-512 could select different kernels. **Re-run the §E gate on the deployment host** — the most important residual risk, and the cheapest to close.

G3 — D1's benefit tracks the length distribution. 26.2% of pairs are already at 512 and can never bucket cheaper; if V2 chunking changes, re-measure. **G5 — E-L10 CE 5903.9 ms is interpolated, not measured** (`run_exp018b.py:697`), so every projection inherits its error.

H — Expected realistic latency range

Ratios only, applied to the stored E-L10 figures. **Projections to verify, not results.**

Scenario	CE ms	E-L10 total	note
current (stored)	5,903.9	6,454.8	—
D1 alone — bucketing, threads untouched	2,810	3,360	machine-independent
D1 + merged CE call	2,600	3,150	machine-independent
D1 + threads matched on an 8-CPU host	1,750–1,850	2,300–2,400	agrees with Grok's ≈2380
fast=True on a 4-CPU host	4,840	5,390	the trap

RECOMMENDATION

D1 alone: CE ~5904 -> ~2810 ms, E-L10 ~6455 -> ~3360 ms (~1.9x), bitwise identical, machine-independent. Treat threads as a separate host-measured decision.